

MCRE: A Unified Framework for Handling Malicious Traffic With Noise Labels Based on Multidimensional Constraint Representation

Qingjun Yuan^{1b}, Gaopeng Gou, Yanbei Zhu, Yuefei Zhu^{2b}, Gang Xiong, and Yongjuan Wang^{1b}

Abstract—Due to the limitations of the existing annotation methods, the prevalence of label noise can be caused in realistic malicious traffic datasets, which has a significant impact on the training and evaluation of deep learning-based intrusion detection models. Recently, various methods have been proposed to deal with noise-containing labeled datasets, and they can be roughly divided into two categories: data cleaning and robust training. However, the different processing ideas lead these two types of methods to ignore the information in different components of the dataset, resulting in a cliff-like drop in performance under high noise conditions. To this end, this study proposes a unified framework for handling noise malicious traffic based on the multidimensional constrained representations named MCRE, which unifies data cleaning and robust training into an ideal representation function approximation. According to the properties of the ideal representation function, information integrity constraints, cluster separability constraints and core proximity constraints are defined to drive MCRE to approximate the ideal representation during iteration. These constraints led MCRE to learn the individual, intra-class, and global levels of distributed knowledge, thus avoiding irrational domain knowledge extraction and ensuring strong label noise robustness of the representation network. We validated MCRE on a dataset that includes 22 types of realistic malicious traffic. Experimental results show that MCRE can outperform the state-of-the-art methods in both data cleaning and robust training downstream tasks, achieving 85% pure sample rate and 82% classification accuracy even under the condition of up to 90% noise labels. In addition, the generalizability of MCRE was verified on several public datasets. Finally, MCRE was also well-extended to enhance other data cleaning and robust training approaches.

Index Terms—Malicious traffic detection, noise labels, data cleaning, robust training, deep learning.

I. INTRODUCTION

TRAINING models using realistic malicious traffic is considered an effective way to reduce the impact

of data cascading [1] on deep learning-based intrusion detection systems (IDS). However, malicious traffic often uses encryption and redundancy to obfuscate itself [2], so annotators cannot accurately characterize the behavior of novel attacks (i.e., 0day attacks), making the existing annotation methods prone to mislabeling [3].

However, deep learning models are highly dependent on large-scale, accurate-labeled data [4], [5], [6], [7]. The noisy labels in real traffic datasets make them difficult to be used for intrusion detection models [8]. To this end, various approaches have been proposed to handle noisy labels, and they can be roughly divided into two categories: **data cleaning** [9], [10], [11], [12] and **robust training** [13], [14], [15], [16]. These two categories differ in their objectives, as shown in Tab. I. Particularly, data cleaning aims to output a purer dataset, while robust training aims to output an accurate classification model directly.

In the above approaches, optimization constraints consistent with the task objectives are usually designed to help the network achieve the preset goal, which could be referred to as task-guided training. Such task-guided training allows the network to ignore part of the domain knowledge in the dataset, resulting in locking the model potential [17]. The details are as follows:

In general, in order to obtain a purer dataset, **data cleaning** focuses more on simple samples, resulting in a large number of hard samples being discarded, making the distributional properties of the cleaned dataset worse. For instance, INCV [9] selects simple samples to update the network in multiple iterations, and ULDC [12] determines the label confidence of hard samples based on the locations of simple samples, but is prone to misclassifying and discarding hard samples. Such discarding of hard samples leads to poor generalization of classifiers trained on cleaned datasets [12].

In contrast, **robust training** focuses more on acquiring global domain knowledge from hard samples, resulting in a model with high accuracy [17]. For example, Co-teaching+ [15] selects the hard samples with disagreement in the iterations to update network parameters efficiently and prevent performance degradation. Unicon [19], on the other hand, selects hard samples by the performance of simple samples in the iteration. However, the models obtained by the robust training approach are generally affected by noise labels, and the accuracy of classifiers is generally poor when the noise level is high [12].

Manuscript received 26 May 2023; revised 29 August 2023 and 14 September 2023; accepted 18 September 2023. Date of publication 25 September 2023; date of current version 20 November 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB3101400 and in part by the Blockchain System Security Key Technology Research of Henan Province Major Public Welfare Project under Grant 2013002102000. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Ghassan Karame. (Corresponding author: Yongjuan Wang.)

Qingjun Yuan, Yanbei Zhu, Yuefei Zhu, and Yongjuan Wang are with the Henan Key Laboratory of Network Cryptography Technology, Zhengzhou 450001, China (e-mail: pinkywyj@163.com).

Gaopeng Gou and Gang Xiong are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100080, China.

Digital Object Identifier 10.1109/TIFS.2023.3318962

TABLE I
COMPARISON OF TWO WAYS TO HANDLE NOISE LABELS

	Representative work	Task description	Objectives
Data cleaning	INCV [9], CL [10], FINE [11], ULDC [12]	Select pure samples from a noise-labeled dataset or remove samples suspected of being noisy to form a purer dataset	Cleaned dataset
Robust training	MentorNet [13], Co-teaching [14], Co-teaching+ [15], Co-learning [16]	Train a model that can accurately identify the same distributed traffic using a training set containing noisy labels	Classification models

To solve the aforementioned problem, a simple idea is to transform the task-guided training process [9], [10], [11], [12], [13], [14], [15], [16] into an objective attribute-guided process, thus avoiding selective learning of knowledge. We refer to attribute-guided process, which is a method to guide the updating of neural network parameters by measuring the difference in attributes between that neural network and an ideal data cleaning and robust training model. During training, the neural network is guided to approximate the ideal model by narrowing the differences in attributes from the ideal target, making the resulting neural network suitable for both data cleaning and robust training tasks. We theoretically analyzed and summarized the attributes of ideal data cleaning and robust training models, and translated them into a unified attribute-guided framework.

Specifically, we propose **MCR**e, a unified framework based on **M**ultidimensional **C**onstraint **R**epresentation that is compatible with both data cleaning and robust training tasks. MCR*e* theoretically transforms the problems of data cleaning and robust training into a unified problem of approximating the ideal representation function of traffic, which can be simply formulated as follows.

$$\mathcal{G} \rightarrow \mathcal{G}_{idl}. \quad (1)$$

By ideal representation $\mathcal{G}_{idl}$, we mean a function that can perfectly reconstruct samples in a low-dimensional decision space and can project samples belonging to the same class to similar positions on the decision space, thus achieving ideal data cleaning or robust classification. To enable the representation network $\mathcal{G}$ to approximate $\mathcal{G}_{idl}$ in iterations, three types of attribute constraints are defined, namely information integrity constraint, cluster separability constraint, and core proximity constraint, based on the nature of the ideal representation function. In this way, a unified and efficient framework for malicious traffic data cleaning and robust training is designed. In training, the three attribute constraints collaborate to guide the network parameters update, where the cluster separability constraint and the core proximity constraint focus on learning from difficult and simple samples, respectively, helping MCR*e* to maintain higher performance under high noise conditions.

To the best of our knowledge, MCR*e* is the first framework that combines data cleaning and robust training. The main contributions are as follows:

- We proposed a unified framework for handling the noise labels of malicious traffic, MCR*e*, consisting of a representation solver and two downstream tasks. MCR*e* first represented all traffic uniformly as vectors that are robust enough to label noise, and then designed a

distance-based data cleaning and training algorithm based on vector distribution in the decision space.

- We proposed a novel perspective to solve the problem of handling noise labels: attribute-guided ideal representation approximation. We theoretically analysed the attributes of the ideal function and designed three different levels of constraints to help the representation network learn individual, intra-class and global distributions simultaneously, guiding the representation network to approximate the ideal representation during iteration, thus achieving a robust representation of the traffic.
- Numerical results showed that MCR*e* outperforms other state-of-the-art approaches in data cleaning and robust classification model training under high noise conditions. In addition, MCR*e* also had good generalisability and extensibility.

The rest of this paper is organized as follows. In Section II, we discuss the current status of noisy labels in traffic data and summarize existing methods for handling noisy labels. In Section III, we analyze the attributes of the ideal representation function theoretically and introduce the three constraints we proposed. In Section IV, we detail the proposed MCR*e*. In Section V, we present the experiments and their results. Finally, we conclude the paper in Section VI.

II. RELATED WORK

A. Noise Labels in Traffic Datasets

Training intrusion detection models that use realistic malicious traffic can ensure better model generalization and has become a common approach in the industry. However, due to the limited accuracy of existing annotation methods [20], the completeness of annotation strategies [21] and the misperception of traffic distribution by different annotators [22], the dataset is prone to mislabeling. In addition, frequent zero-day attacks pose a severe challenge to the accurate labeling of realistic traffic [23]. These attacks are beyond the knowledge of annotators and annotation systems, so traffic is often difficult to label correctly. Similarly, variants of known attacks, due to variations in certain attributes, can easily interfere with the judgment of rule-based or IDS-based automatic labeling systems.

Mislabeling is common in traffic datasets collected from real networks. Models trained with such data are prone to performance degradation during online detection, with a possibility of generating a large number of false positives and misses, limiting the application of deep learning models to realistic IDS [24]. Specifically, the impacts on real systems can be summarized as follows:

Model Evaluation and Selection: Evaluation of deep learning models requires accurately labeled traffic that has the same or similar distribution to the target environment [25]. Traffic labelled incorrectly or obtained from isolated environments cannot accurately assess and evaluate model performance, so correct feedback cannot be provided to model evaluators, which results in inappropriate models coming online. Therefore, it is crucial to clean noisy traffic dataset and obtain a purer dataset that can help IDS to assess and evaluate the models accurately.

Domain Knowledge Extraction: Extracting domain knowledge also requires accurately labeled traffic. Inaccurately labeled datasets can easily drive the model to learn something wrong during iterations, ignoring some important features or knowledge and thus affecting subsequent analysis and predictions. However, models trained on complex and diverse traffic collected in real networks have superior generalization ability compared with models trained on closed datasets. Therefore, it is also an important task to train accurate and robust classifier models on realistic traffic data containing noise labels.

B. Noisy Labels Handling

Inaccurate ground truth and non-stationary distribution of traffic make it difficult for deep learning-based IDSs to identify malware, leading to various cyber security incidents [8]. Therefore, it has become important for the real world to deal with noisy labels in traffic datasets to form cleaner datasets or to train robust traffic identification models. Depending on the objectives, the existing methods can be divided into two categories.

1) *Data Cleaning:* Distinguish noise and pure samples in the traffic dataset by identifying and evaluating instances and labels, and output a dataset consisting of pure samples. The evaluation of labels is mainly performed by measuring the consistency in features of the samples [26] or density distribution [10], [11] after the representation with the observed labels. Although the above approaches have been successful in reducing the proportion of noise labels in the dataset, the data cleaning process often discards a large number of outlier samples as well. However, the absence of these samples can shift the global distribution, making it difficult to accurately assess the generalisability of models with the cleaned dataset.

2) *Robust Training:* Reduce the impact of untrustworthy labels on model parameter updating by selective learning of the global distribution, resulting in more accurate classifiers. The performance of these models is improved mainly by designing robust loss functions [27], [28] or network architectures [14], [15], [19] that prevent supervised neural networks from overfitting the noisy labels. However, when noise levels are high, these models can struggle to acquire correct domain knowledge, making it difficult to obtain positive feedback on model parameter updates.

Due to the different objectives, there are certain differences in the way the above approaches deal with noise labels. For data cleaning, in order to improve the purity of the dataset, more emphasis is often put on screening individuals

and excluding samples that cannot be identified. In contrast, for robust training, noise samples are introduced more aggressively during model training to ensure the generalization of the model and accurate decision boundaries, and the impact of noise samples is mitigated by relying on the robustness of the model itself [29]. However, knowledge selectivity leads to a dramatic drop in performance when these approaches face high noise levels.

To this end, we propose a unified noise processing framework named MCR, which converts the problems of data cleaning and robust training into an ideal function approximation problem from an alternative perspective. In addition, cluster separability and core proximity constraints for MCR are defined to help the representation network learn individual and global knowledge in iterations, respectively, thus maintaining good performance of MCR under high-noise conditions.

III. PROBLEM DESCRIPTION & ATTRIBUTE CONSTRAINTS DESIGN

In this section, the problem of representing traffic with noisy labels is formulated.

Consider a C -classes traffic dataset with noisy labels and let $\mathbf{D} := \{(\mathbf{x}_i, \tilde{y}_i)\}^N$ be a traffic dataset containing noisy labels, where $(\mathbf{x}_i, \tilde{y}_i)$ is a labeled sample, $\mathbf{x}_i$ represents the i -th sample and $\tilde{y}_i$ is the observed label corresponding to $\mathbf{x}_i$. For each $\mathbf{x}_i$, there is one and only one latent true label denoted by y_i^* , and $\tilde{y}_i, y_i^* \in \{1, 2, \dots, C\}$.

The main goal is to derive a representation function $\mathcal{G}$ that can reconstruct traffic while creating a useful and meaningful latent representation. $\mathcal{G}$ should avoid using noisy labels and handle noisy labels appropriately, which helps to clean the labels and train powerful classifiers with noisy labels.

First, the ideal representation under noisy labels is defined.

Definition 1: Ideal Representation: For $\mathbf{D}$, the ideal representation is a mapping $\mathcal{G}_{idl}(\mathbf{x}) := \mathbb{R}^m \rightarrow \mathbb{R}^s$ such that for all sample pairs $(\mathbf{x}_i, \mathbf{x}_j)$, there exists a constant $\varepsilon > 0$ that satisfies the following condition:

$$d(\mathcal{G}_{idl}(\mathbf{x}_i), \mathcal{G}_{idl}(\mathbf{x}_j)) \leq \varepsilon \Leftrightarrow y_i^* = y_j^*, \quad (2)$$

where d is a function that measures the distance between two samples. Common distance measurement functions include l_1 , l_2 , l_p , l_∞ , and cosine distance.

Traffic belonging to the same category, with similar behavioral patterns, is distributed with certain a similarity in the feature space. Under ideal conditions, the mapping distance in the decision space is closer ($\leq \varepsilon$) for samples with the same true labels, but farther ($> \varepsilon$) for samples that do not belong to the same class. Furthermore, ε is used as a boundary to divide clusters in the decision space, thus forming an ideal decision boundary. Obviously, in this ideal situation, accurate identification of all individuals in the dataset can be achieved, realizing ideal data cleaning.

Therefore, to construct a function $\mathcal{G}$ that is as close to $\mathcal{G}_{idl}$ as possible, we propose three conditional constraints from a mathematical perspective to help $\mathcal{G}$ converge towards $\mathcal{G}_{idl}$ during training.

Constraint 1 Information Integrity Constraint: For any $\mathbf{x}_i \in \mathbf{D}$, there is

$$\mathcal{G} \leftarrow \arg \min_{\mathcal{G}} (H(\mathbf{x}_i) - H(\mathcal{G}(\mathbf{x}_i))), \mathbf{x}_i \in \mathbf{D}, \quad (3)$$

where $H(\cdot)$ represents information entropy.

We argue that a good representation function should be as informative as possible. Generally, however, the represented traffic, which has lower dimensions, prompts the neural network to learn the most informative features. In practice, the dimension of the representation is usually set manually as a hyperparameter. As $\mathcal{G}$ compresses the dimensionality of traffic, some information will be inevitably lost.

Constraint 2 Cluster Separability Constraint: For any $\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k \in \mathbf{D}$, if $y_i^* = y_j^* \neq y_k^*$, then it holds that

$$d(\mathcal{G}(\mathbf{x}_i), \mathcal{G}(\mathbf{x}_j)) \leq d(\mathcal{G}(\mathbf{x}_i), \mathcal{G}(\mathbf{x}_k)). \quad (4)$$

The cluster separability constraint is essentially a variant of the ideal representation. By Eq. 2, it is clear that $y_i^* \neq y_k^* \Leftrightarrow d(\mathcal{G}(\mathbf{x}_i), \mathcal{G}(\mathbf{x}_k)) \geq \varepsilon$. Together with $y_i^* = y_j^* \Leftrightarrow d(\mathcal{G}(\mathbf{x}_i), \mathcal{G}(\mathbf{x}_j)) \leq \varepsilon$, it is easy to obtain Eq. 4.

A good representation function should be able to reconstruct samples belonging to the same class as individuals with spatial similarity in the decision space, making them far from the other categories, which in turn helps to construct well-distinguished decision boundaries.

Constraint 3 Core Proximity Constraint: For every $\mathbf{x}_i \in \mathbf{D}$, there exists a virtual full confidence sample $\mathbf{x}^{(k)}$, such that

$$\mathcal{G} \leftarrow \arg \min_{\mathcal{G}} \sum_{k=1}^C \sum_{\mathbf{x}_i \text{ where } y_i^* = k} d(\mathcal{G}(\mathbf{x}_i), \mathcal{G}(\mathbf{x}^{(k)})) \quad (5)$$

A full confidence sample is a sample that is credibly attributed to a specified class. A good representation function should make each sample as close as possible to the projection of the full confidence sample of the class, so that the samples are better aggregated in the decision space.

We believe that to construct a suitable representation function, the above three constraints must be satisfied, or satisfied as far as possible.

IV. ARCHITECTURE OF MCRE

The architecture of MCRE is shown in Fig. 1 and consists of a noise traffic characterization network and two corresponding downstream task layers. The former is the central component where MCRE constructs a mapping function $\mathcal{G}$ that satisfies the multidimensional constraints. The representation network should have the following properties: 1) feature representations should retain as much input information as possible; 2) samples with the same true labels are should be closer to each other while farther away from classes with different labels; and 3) samples with the same true labels should be closer to the full confidence samples within the class. In order to help the representation network obtain the above properties, we design the corresponding constraint generating networks to measure and adjust the current representation network, and optimize the parameters of the feature representation function by back propagation. As for the two downstream tasks of data cleaning and robust training, the distance-based cleaning

and classification layers are connected to the representation network and the corresponding dataset or classification model is output. In this section, the details and meaning of these components will be explained.

A. Deep Representation

The goal of representation is to reconstruct traffic in a low-dimensional decision space, aggregating similar samples and separating dissimilar samples to provide a more intuitive view for downstream tasks, such as traffic cleaning and identification. In this study, a 7-layer, fully-connected encoder is used as the deep representation. It has a simple structure and strong plasticity, and is more receptive to feedback from the three constraints.

The deep representation is defined as follows.

$$\mathbf{u}_i = \mathcal{G}(\mathbf{x}_i; \theta_{\mathcal{G}}), \quad (6)$$

where $\theta_{\mathcal{G}}$ represents the parameters corresponding to $\mathcal{G}$ and $\mathbf{u}_i$ represents the representation of $\mathbf{x}_i$ in the decision space.

The feedback of each of the three constraints on the network is determined by measuring the distribution of $\mathbf{u}$ in the decision space. In each iteration, $\mathcal{G}$ will update $\theta_{\mathcal{G}}$ according to the integration loss generated by the three constraints, as follows.

$$\theta_{\mathcal{G}} \leftarrow \theta_{\mathcal{G}} - \eta \nabla \mathcal{L}(\mathbf{D}; \theta_{\mathcal{G}}), \quad (7)$$

where η denotes the learning rate and $\mathcal{L}$ is defined by:

$$\mathcal{L} = \mathcal{L}_{iic} + \mathcal{L}_{csc} + \mathcal{L}_{cpc} + \mathcal{L}_{cls}, \quad (8)$$

where $\mathcal{L}_{iic}$ is given by Eq. 19, $\mathcal{L}_{csc}$ is defined by Eq. 26, $\mathcal{L}_{cpc}$ is obtained by Eq. 30, and

$$\mathcal{L}_{cls} = CE(\tilde{\mathbf{y}}, \mathcal{F}(\mathbf{u})), \quad (9)$$

where CE denotes the cross-entropy loss, and $\mathcal{F}$ is a fully connected classifier head. It has been shown that the representation network before the classification head does not significantly overfit to the label noise [30], [31]. Therefore, the use of noise labels (or even random labels [32]) can also have a facilitating effect on the representation network.

B. Network of Information Integrity Constraint

As given in Eq. 3, the information integrity constraint requires that the transformations have as small information lost as possible. To satisfy the information integrity constraint, we design a decoding network $\mathcal{G}^{-1}$ that is symmetric with the representation network, and it forms a classic self-encoder structure with the representation network.

For a given $\mathcal{G}$ and $\mathcal{G}^{-1}$, the information reduced by $\mathcal{G}$ can be quantified as

$$H(\mathbf{x}) - H(\mathbf{u}) = I(\mathbf{x}; \mathbf{u}) - H(\mathbf{u}|\mathbf{x}), \quad (10)$$

where $I(\mathbf{x}; \mathbf{u})$ denotes the mutual information between $\mathbf{u}$ and $\mathbf{x}$.

And since both $\mathbf{x}$ and $\mathcal{G}$ are deterministic, $H(\mathbf{u}|\mathbf{x}) = 0$, so

$$H(\mathbf{x}) - H(\mathbf{u}) = I(\mathbf{x}; \mathbf{u}), \quad (11)$$

Further, it can be obtained that

$$H(\mathbf{u}) - H(\mathcal{G}^{-1}(\mathbf{u})) = I(\mathbf{u}; \mathcal{G}^{-1}(\mathbf{u})). \quad (12)$$

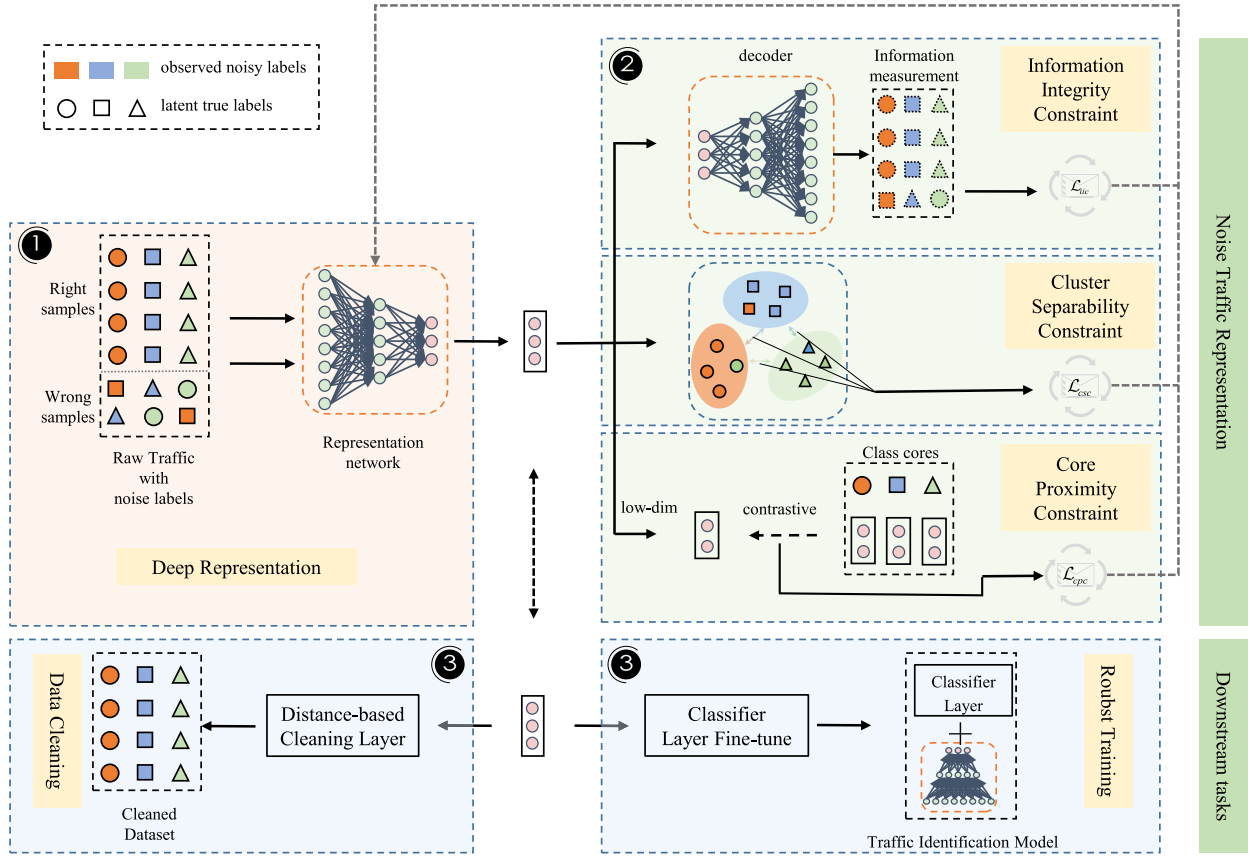


Fig. 1. Architecture of MCR.

Then

$$H(\mathbf{x}) - H(\mathcal{G}^{-1}(\mathbf{u})) = I(\mathbf{x}; \mathbf{u}) + I(\mathbf{u}; \mathcal{G}^{-1}(\mathbf{u})). \quad (13)$$

In addition, since $I(\cdot) \geq 0$, it can be written that

$$\begin{aligned} \min (H(\mathbf{x}) - H(\mathcal{G}^{-1}(\mathbf{u}))) \\ &= \min (I(\mathbf{x}; \mathbf{u})) + \min (I(\mathbf{u}; \mathcal{G}^{-1}(\mathbf{u}))) \\ &= \min (H(\mathbf{x}) - H(\mathbf{u})) + \min (I(\mathbf{u}; \mathcal{G}^{-1}(\mathbf{u}))). \end{aligned} \quad (14)$$

Therefore, $H(\mathbf{x}) - H(\mathcal{G}^{-1}(\mathbf{u}))$ has a minimal value when and only when both $H(\mathbf{x}) - H(\mathbf{u})$ and $I(\mathbf{u}; \mathcal{G}^{-1}(\mathbf{u}))$ have their minimum values. Thus, it can be written that

$$\arg \min_{\mathcal{G}, \mathcal{G}^{-1}} (H(\mathbf{x}) - H(\mathcal{G}^{-1}(\mathbf{u}))) = \arg \min_{\mathcal{G}, \mathcal{G}^{-1}} (H(\mathbf{x}) - H(\mathbf{u})). \quad (15)$$

In addition, since $\mathcal{G}$ and $\mathcal{G}^{-1}$ are symmetric, $\mathcal{G}^{-1}(\mathbf{u})$ can be considered as an estimate of the distribution of $\mathbf{x}$. Therefore, it holds that

$$\arg \min_{\mathcal{G}, \mathcal{G}^{-1}} (H(\mathbf{x}) - H(\mathbf{u})) = \arg \min_{\mathcal{G}, \mathcal{G}^{-1}} (H(\mathbf{x}, \mathcal{G}^{-1}(\mathbf{u})) + H(\mathbf{x})) \quad (16)$$

Again, since $H(\mathbf{x})$ is necessarily >0 and for a given $\mathbf{D}$, the value of $H(\mathbf{x})$ is constant.

$$\arg \min_{\mathcal{G}, \mathcal{G}^{-1}} (H(\mathbf{x}) - H(\mathbf{u})) = \arg \min_{\mathcal{G}, \mathcal{G}^{-1}} (H(\mathbf{x}, \mathcal{G}^{-1}(\mathbf{u}))). \quad (17)$$

In summary, the goal of the information integrity constraint is to minimize the value of Eq. 17, which

$$\mathcal{G} \leftarrow \arg \min (H(\mathbf{x}, \mathcal{G}^{-1}(\mathbf{u}))). \quad (18)$$

However, for traffic vectors in a high-dimensional space, calculating their cross-entropy is a challenging task; therefore, in this study, the Binary Cross-Entropy is adopted, and the information integrity loss of samples in each iteration is calculated by

$$\begin{aligned} \mathcal{L}_{iic}(\mathbf{D}; \theta_{\mathcal{G}}, \theta_{\mathcal{G}^{-1}}) \\ &= \frac{1}{|\mathbf{D}|} \sum_{i=1}^{|\mathbf{D}|} \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log (1 - \hat{x}_{ij})), \end{aligned} \quad (19)$$

where x_{ij} is the j -th component of $\mathbf{x}_i$, and $\hat{\mathbf{x}} = \mathcal{G}^{-1}(\mathcal{G}(\mathbf{x}))$.

C. Network of Cluster Separability Constraint

According to Eq. 4, the goal of the cluster separability constraint is to separate the mixed samples near the cluster boundaries to form clear and accurate decision boundaries. Therefore, as shown in Fig. 2, the representation function should achieve two goals: (1) compress the distance between samples attributed to the same cluster, reducing the radius of the cluster; (2) expand the distance between samples attributed to different clusters, increasing the distance between clusters.

First, the samples are divided into different clusters in the decision space based on their projection distances, which are denoted by $\{clus_1, clus_2, \dots, clus_C\}$. Similar to FINE [11],

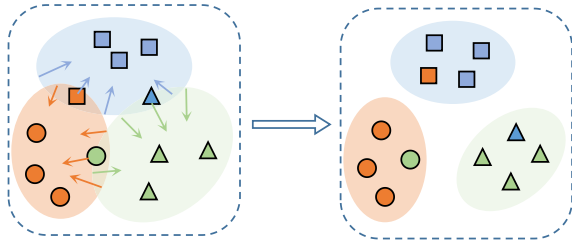


Fig. 2. Cluster separability constraint.

the Euclidean distance is selected in this study to measure the separability. The distance between $\mathbf{x}_i$ and $\mathbf{x}_j$ is calculated by

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{u}_i - \mathbf{u}_j\|_2 \quad (20)$$

To achieve the first goal, the average distance of each sample to other samples in the same cluster is calculated. Assume that $\mathbf{x}_i \in \text{clus}_k$; then, the average distance from $\mathbf{x}_i$ to the other samples in the cluster can be expressed as follows:

$$d_{\mathbf{x}_i}^{\text{in}} = \frac{1}{|\text{clus}_k| - 1} \sum_{\mathbf{x}_j \in \text{clus}_k} d(\mathbf{x}_i, \mathbf{x}_j). \quad (21)$$

To achieve the second goal, the closest distance of each sample to the other cluster boundaries is calculated. For computational purposes, we calculate the distance from the sample to the nearest sample that does not belong to the cluster. Then the distances to other clusters can be expressed as

$$d_{\mathbf{x}_i}^{\text{out}} = \min \{d(\mathbf{x}_i, \mathbf{x}_j) \mid \mathbf{x}_j \notin \text{clus}_k\}. \quad (22)$$

From a practical perspective, using absolute distances to measure cluster separability is not completely consistent with the goal of formulating an ideal representation. Therefore, the two goals are unified as a ratio of the difference to measure to the deviation of each sample from the ideal representation, as follows:

$$\delta_{\mathbf{x}_i} = \frac{d_{\mathbf{x}_i}^{\text{out}} - d_{\mathbf{x}_i}^{\text{in}}}{d_{\mathbf{x}_i}^{\text{out}}} \quad (23)$$

where $\delta_{\mathbf{x}_i}$ denotes the difference between the distance of $\mathbf{x}_i$ to the other clusters and the distance to the current cluster; it indicates how easily $\mathbf{x}_i$ can be correctly distinguished under the representation of $\mathcal{G}$. In order to ensure convergence of the above equation, it is normalized as follows

$$\delta_{\mathbf{x}_i}^{\text{norm}} = \text{Norm}(\delta_{\mathbf{x}_i}). \quad (24)$$

Therefore, at each iteration, the cluster separability constraint for $\mathcal{G}$ can be expressed by

$$\mathcal{G} \leftarrow \arg \max \left(\frac{1}{|\mathbf{D}|} \sum \delta_{\mathbf{x}_i}^{\text{norm}}; \mathcal{G} \right) \quad (25)$$

Furthermore, the loss of cluster separability can be obtained by

$$\mathcal{L}_{\text{csc}}(\mathbf{D}; \theta_{\mathcal{G}}) = 1 - \frac{1}{|\mathbf{D}|} \sum \delta_{\mathbf{x}_i}^{\text{norm}} \quad (26)$$

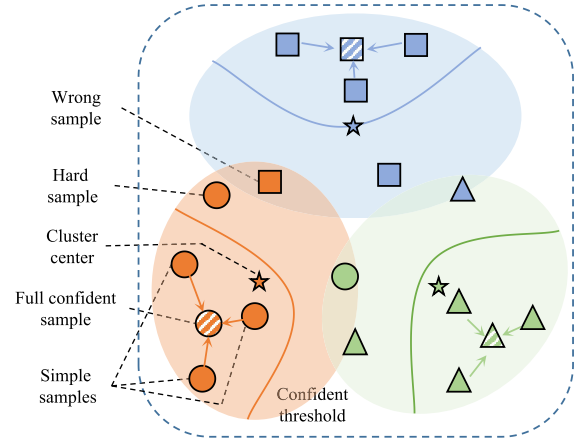


Fig. 3. Core proximity constraint.

D. Network of Core Proximity Constraint

Eq. 5 shows that the core proximity constraint requires $\mathcal{G}$ to project every sample as close as possible to the full confident sample of the class to which the sample truly belongs. Therefore, the representation function should drive the projection of the sample close to its full confidence sample. The corresponding constraint is related to two following steps: full confidence sample selection and constraint generation.

1) *Full Confidence Sample Selection*: A full confidence sample is a sample that is fully identified as belonging to a particular class. In this subsection, we identify the center of the simple sample as the full confidence sample for this class, as shown in Fig. 3. Unlike the unlabeled clustering centers generated in Sec. IV-C, the full confidence samples in this section denote the cores of the classes with observed labels. First, the simple samples are selected in each class according to their distribution in the decision space; then, the full confidence samples in each class are calculated according to the distribution of the simple samples. It has been a common strategy for obtaining label information to avoid the effect of labels on the construction of the representation function under high noise conditions.

Simple samples represent samples that are far from boundaries, belong to a certain class with a high probability, and tend to have a small loss in classification tasks. Co-teaching [14] and Co-teaching+ [15] select simple samples using supervised classifiers, and a similar idea is adopted in this study. Specifically, a fully-connected classifier $\mathcal{F}$ is defined in the decision space, and it is determined whether the sample is a simple sample based on its output as follows.

$$\mathbf{x}_i \in \text{Sim}_k, \text{ if } \tilde{y}_i = \arg \max_k \mathcal{F}_k(\mathbf{u}_i), \quad (27)$$

where Sim_k is the set of simple samples labeled with k , and $\mathcal{F}_k(\mathbf{u}_i)$ is the score classified by $\mathbf{x}_i$ as k . That is, we consider a sample to be a simple sample when the original label has a higher score than the other labels. In fact, there could be some outlier samples classified in Sim_k , but because their number is usually relatively small, they do not affect the classification result.

Further, samples in Sim_k are projected into a lower dimensional space $\mathbf{z}$ to reduce the computational cost of full confidence samples computing and improve the robustness

of sample representations. The second projection on the representation layer has been a common scheme and is widely used in classical models such as MoCo [33]. The experimental results have been available to demonstrate the effectiveness. We use the momentum prototype of all samples in Sim_k as the full confidence sample $\mathbf{z}^k$ of the flow for category k , which is defined by

$$\mathbf{z}^k \leftarrow \text{Norm} \left(m\mathbf{z}^k + (1 - m)\mathbf{z}_i \right), \mathbf{z}_i \in Sim_k, \quad (28)$$

where $\mathbf{z}_i$ is the projection of $\mathbf{u}_i$ in the $\mathbf{z}$ -space. In this paper, m is set to 0.999. In each iteration, the full confidence samples are updated to the mean of weighted samples in the $\mathbf{z}$ -space.

2) *Constraint Generation*: The main goal is to search for an embedding space where samples belonging to the same class cluster are around their full confidence samples; namely, samples closer to $\mathbf{z}^k$ have lower losses when network parameters are updated, thus driving samples towards the full confidence samples.

At each iteration, the core proximity constraint of can be expressed by

$$\mathcal{G} \leftarrow \arg \min_{\mathcal{G}, \mathcal{F}} \frac{1}{|\mathbf{D}|} \sum \left(\min \left\| \mathbf{z}_i - \mathbf{z}^k \right\|_2 \right). \quad (29)$$

Then, in each iteration, the core proximity loss of $\mathcal{G}$ can be quantified as

$$\mathcal{L}_{cpc} = \frac{1}{|\mathbf{D}|} \sum \left(\min CE \left(\mathbf{z}_i, \mathbf{z}^k \right) \right) \quad (30)$$

E. Downstream Tasks

In general, downstream tasks include data cleaning and robust training. In Tab. I, these two tasks are described in detail. For the two tasks, a simple data cleaning method named MCR-dc and a robust training method named MCR-clc are designed based on the distance metric of samples in the decision space constructed by MCR.

1) *MCR-dc*: According to the distance distribution of samples in the decision space constructed by MCR, the suspected noise samples are determined, and sample removal and label correction are performed. In this study, the confidence of labels is calculated using the difference in distance between the samples and the full confidence samples. During data cleaning, the mislabeling of samples with high confidence values is considered, and samples without high confidence attribution classes are discarded.

Specifically, we set a fixed sample discard ratio and discard samples that are far from the full confidence samples (i.e., samples with low label confidence). This is one of the simplest data cleaning methods and is widely adopted [40]. In this paper, we focus on whether MCR based on multidimensional constrained representations has the potential to perform well in data cleaning, and therefore only the simplest fixed threshold scheme is adopted. In practice, more effective data cleaning schemes can be selected based on specific objectives.

2) *MCR-clc*: MCR-clc accesses and trains a classification layer on the representation layer of MCR and then outputs a classification model. In this study, a simple Kmeans classifier is used to divide traffic into clusters. The traffic is identified using the Kuhn-Munkres algorithm, which matches the clusters to their corresponding labels.

V. EXPERIMENTAL ANALYSIS

In this section, the effectiveness and robustness of MCR were verified by comparison with other comparative approaches in the two downstream tasks.

A. Datasets and Noise Settings

1) *Private Dataset*: A dataset containing 22 types of encrypted malicious traffic named Malicious_TLS was constructed to evaluate the performance of MCR. In this dataset, all traffic was encrypted with TLS and thus behaved similarly. Consequently, cleaning and identifying such TLS traffic would also be a more difficult task when noise labels were present. In addition, all malicious traffic was collected from real edge network devices. The traffic was captured over a four-year period from 2018 to 2021 and labeled with precise threat intelligence to ensure Ground Truth. Also, all benign traffic was encrypted using TLS. Note that no public dataset currently contained such a wide variety of realistic malicious encrypted traffic. Anyone can fetch the dataset by visiting https://github.com/gcx-Yuan/Malicious_TLS. In particular, to prevent privacy, we erased IPs, ports, timestamps, payloads, etc. from the traffic. Semantic, statistical and spatio-temporal features of these traffic are also extracted.

2) *Public Datasets*: We also verified the effectiveness of MCR on the CICIDS-2017 [36] intrusion detection dataset. Specifically, rather than extracting features from raw traffic, we use the features offered. In addition, to evaluate the generalizability of MCR, we validated it on several public malicious traffic datasets, including NSL-KDD [34], UNSW-NB15 [35], LITNET-2020 [37], IOT-23 [38], and CICMalDroid-2020 [39].

3) *Noise Settings*: Similar to ULDC [12], we set up two types of scenarios: asymmetric scenarios and symmetric scenarios, based on different traffic labeling strategies in the real world.

Asymmetric noise was usually labeled by multiple independent labeling entities that determine the labels of samples through strategies such as voting. Different labeling strategies will result in different distributions of label noise. Of these, labeling strategies that reduce false positives [24] have been the most common because they maximize the availability of the system. In this case, a large amount of malicious traffic will be labeled as benign, and only a small amount of benign traffic will be labeled as malicious. We defined this as an asymmetric scenario, where only a portion of the malicious traffic labels was flipped to benign.

Symmetrical noise is usually labeled by a single or few entities, and in that case, a large amount of malicious traffic will be labeled as benign, but also, a large amount of benign traffic will be labeled as malicious. The quality of labels depends entirely on the capacity of entities and results in different ratios of noise labels. In short, in symmetric noise scenarios, all traffic labels have a certain probability of flipping.

In this study, the range of noise label ratios was set to [0.1, 0.9] to explore the robustness of different approaches under harsh high noise conditions. Fig. 4 shows the label conversion matrix for the 40% noise ratio case for both scenarios. The horizontal axis represents observed noise labels

TABLE II
PERCENTAGE OF PURE SAMPLES IN ASYMMETRIC SCENARIOS ON MALICIOUS_TLS

Percentage of pure samples	10%	20%	30%	40%	50%	60%	70%	80%	90%
INCV	98.26 $\pm$ 0.73	90.21 $\pm$ 0.97	83.50 $\pm$ 1.62	78.78 $\pm$ 1.91	56.61 $\pm$ 2.75	50.77 $\pm$ 2.64	45.75 $\pm$ 3.04	42.25 $\pm$ 3.75	39.45 $\pm$ 4.19
CL	99.29 $\pm$ 0.69	94.65 $\pm$ 1.25	81.50 $\pm$ 1.33	69.41 $\pm$ 2.34	61.08 $\pm$ 2.50	51.71 $\pm$ 2.75	50.93 $\pm$ 3.56	49.96 $\pm$ 3.37	40.89 $\pm$ 4.66
FINE	98.00 $\pm$ 0.85	91.53 $\pm$ 1.25	84.23 $\pm$ 1.85	75.67 $\pm$ 1.76	61.39 $\pm$ 2.25	50.41 $\pm$ 2.53	48.93 $\pm$ 3.12	40.05 $\pm$ 3.32	30.64 $\pm$ 4.72
ULDC	97.31 $\pm$ 0.99	92.64 $\pm$ 1.08	83.89 $\pm$ 1.93	82.03 $\pm$ 2.26	78.58 $\pm$ 2.53	77.50 $\pm$ 2.07	67.89 $\pm$ 3.26	63.97 $\pm$ 3.53	54.00 $\pm$ 4.07
MCRc-dc (ours)	97.06 $\pm$ 0.14 (2.23 $\downarrow$)	95.84 $\pm$ 0.27 (1.19 $\uparrow$)	95.70 $\pm$ 0.29 (1.147 $\uparrow$)	94.26 $\pm$ 0.48 (12.23 $\uparrow$)	93.16 $\pm$ 0.69 (14.58 $\uparrow$)	92.77 $\pm$ 0.68 (15.27 $\uparrow$)	90.93 $\pm$ 0.91 (23.04 $\uparrow$)	88.71 $\pm$ 1.29 (24.74 $\uparrow$)	86.95 $\pm$ 2.35 (32.95 $\uparrow$)

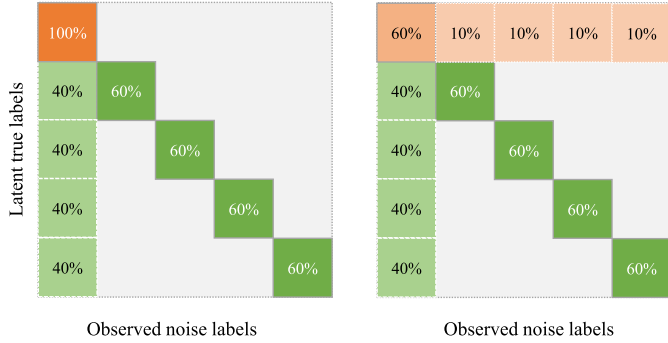


Fig. 4. Label conversion matrix of 40% asymmetric (left) and symmetric (right) noise.

and the vertical axis represents latent true labels. The first row and column are labeled as benign and the other rows and columns are labeled as certain malicious traffic. The left is the asymmetric noise scenario, where 40% of the samples in each class of malicious traffic are randomly selected and their labels are set to benign. The right represents the symmetric noise scenario, where in addition to flipping the malicious traffic labels as in the asymmetric noise scenario, 40% of the benign traffic is selected and its label is randomly set to certain kind of malicious traffic.

4) *Experimental Setup*: We evaluated the performance of two typical downstream tasks on the datasets injected with noise. In the data cleaning task, the entire dataset was injected with noise and then fed to MCRc to output a dataset with a reduced percentage of noise. In the robust training task, the dataset was divided into training and validation sets according to the ratio of 7:3; labels of some of the sample labels in the training set were flipped and fed to MCRc for training; meanwhile, the samples in the validation set were not involved in training and did not contain noise labels, so they were used only to evaluate the performance of the output classifier. In all experiments, five independent experiments were conducted, in which the training and validation sets were randomly divided. Specifically, in each experiment, the average of the final five epochs was taken as the ultimate result.

All experiments were carried out on the same platform equipped with an Intel 9700K@3.60GHz, 32GB RAM, and an NVIDIA GeForce RTX 3090.

B. Experimental Analysis of Data Cleaning

MCRc-dc cleaned the dataset based on the projected distance of the traffic in the decision space. First, MCRc-dc

calculated the distances of the samples from the full confidence sample in each class on the decision space. Then, based on the distance ratio, the confidence of the labels was calculated. Finally, a fixed threshold was set, and all samples with confidence below the threshold were removed.

MCRc-dc was compared with the following state-of-the-art data cleaning methods:

- INCV [9]: INCV trained a supervised classifier on the noisy dataset, and then cleared the samples which the classifier determined were different from their observed labels;
- CL [10]: CL trained a supervised classifier on the noisy dataset and judged the confidence of each sample label based on the output of the classifier, and then removed the samples with low confidence;
- FINE [11]: FINE was based on metric learning and removed samples suspected of having noisy labels;
- ULDC [12]: ULDC was based on an unsupervised neural network that evaluated the confidence of sample labels and removed samples with low confidence.

Tab. II, III, IV and V show the performance of MCRc-dc for cleaning datasets with different noise ratios in asymmetric and symmetric scenarios on Malicious_TLS and CICIDS-2017, respectively. Based on the results, the following conclusions could be drawn:

1. *Of all the approaches, MCRc-dc achieved the best noisy label cleaning results*: Looking longitudinally at each column, there was a significant advantage in the percentage of pure data in the dataset cleaned by MCRc-dc when the noise level was above 20%. In particular, the percentage of pure data was above 85% in both scenarios when the noise level reached 90%, which was more than 25% higher than results of the other approaches. In the other comparative approaches, (a) INCV, CL, and FINE's effectiveness depended on the performance of the supervised classifier, again was limited by the label quality; (b) although ULDC was based on an unsupervised network, its label confidence assessment process relied on label information. As a result, their performance dropped significantly at high noise levels. In contrast, MCRc employed a conservative label trust strategy to ensure stable performance.

2. *Of all the approaches, MCRc-dc exhibited the best robustness*: Looking at each row of the table horizontally, all the approaches reduced the effectiveness of data cleaning as the percentage of noise increased. Of these, MCRc-dc showed

TABLE III
PERCENTAGE OF PURE SAMPLES IN SYMMETRIC SCENARIOS ON MALICIOUS_TLS

Percentage of pure samples	10%	20%	30%	40%	50%	60%	70%	80%	90%
INCV	97.64±0.74	89.91±1.73	80.57±1.66	76.75±2.01	54.39±2.66	49.53±2.75	45.25±3.73	42.81±4.75	36.10±5.98
CL	98.89 ±0.93	81.88±1.32	78.74±1.56	67.84±1.55	56.09±1.92	46.18±2.02	42.16±2.85	39.97±2.91	38.15±3.97
FINE	96.99±0.54	79.11±2.78	70.43±2.69	69.92±3.16	48.23±3.62	40.13±4.33	29.70±5.12	25.44±5.18	19.60±5.26
ULDC	90.96±0.36	92.64±0.99	82.95±1.06	81.68±1.35	70.54±1.77	69.28±2.58	63.53±2.86	57.55±3.82	49.76±3.57
MCRc-dc (ours)	97.46 ±0.17 (1.43↓)	96.59 ±0.21 (3.95↑)	94.62 ±0.24 (11.67↑)	93.84 ±0.55 (12.16↑)	93.85 ±0.70 (23.31↑)	91.16 ±0.95 (21.88↑)	91.11 ±1.37 (27.58↑)	89.03 ±1.93 (31.48↑)	85.72 ±2.96 (35.96↑)

TABLE IV
PERCENTAGE OF PURE SAMPLES IN ASYMMETRIC SCENARIOS ON CICIDS-2017

Percentage of pure samples	10%	20%	30%	40%	50%	60%	70%	80%	90%
INCV	98.42±0.82	92.05±0.87	85.86±1.68	77.96±2.29	59.91±2.53	50.52±2.99	47.91±3.17	45.44±3.77	41.49±4.89
CL	99.51 ±0.15	96.83±0.59	86.65±1.24	79.79±1.89	66.98±2.77	57.25±3.17	55.11±3.95	54.54±4.53	50.53±4.73
FINE	98.48±0.25	92.23±1.06	88.54±1.48	75.97±1.74	63.56±3.01	59.77±3.31	54.05±3.85	49.48±4.53	33.94±5.34
ULDC	97.13±0.23	94.82±0.93	90.53±0.86	87.50±1.85	83.36±2.53	80.72±2.81	78.97±2.64	69.19±3.97	61.00±4.32
MCRc-dc (ours)	99.09 ±0.22 (0.42↓)	98.06 ±0.37 (1.23↑)	97.26 ±0.42 (6.73↑)	95.41 ±0.50 (7.91↑)	94.71 ±0.49 (11.35↑)	93.73 ±1.73 (13.01↑)	90.42 ±2.88 (11.45↑)	89.72 ±2.86 (20.53↑)	88.44 ±3.59 (27.44↑)

TABLE V
PERCENTAGE OF PURE SAMPLES IN SYMMETRIC SCENARIOS ON CICIDS-2017

Percentage of pure samples	10%	20%	30%	40%	50%	60%	70%	80%	90%
INCV	98.78±0.31	91.85±0.94	83.87±1.38	79.74±1.75	64.19±2.23	53.67±2.81	49.90±3.29	44.35±4.63	37.14±5.71
CL	99.52 ±0.45	93.27±1.44	80.56±1.61	76.50±1.68	69.87±2.65	61.46±2.69	57.31±3.46	54.66±4.21	49.42±4.38
FINE	99.01±0.13	89.11±0.75	83.53±1.63	74.16±1.99	68.53±3.11	57.18±3.27	54.47±3.69	47.19±4.78	31.75±5.85
ULDC	97.22±0.34	95.73±0.88	91.25±0.82	89.31±1.29	85.59±2.04	82.93±2.26	80.59±3.45	74.92±3.62	60.54±4.53
MCRc-dc (ours)	99.31 ±0.09 (0.21↓)	98.30 ±0.23 (2.57↑)	97.07 ±0.29 (5.82↑)	97.68 ±0.38 (8.37↑)	94.55 ±0.75 (8.96↑)	93.28 ±0.95 (10.35↑)	92.84 ±1.47 (12.25↑)	89.88 ±2.66 (14.96↑)	87.42 ±3.16 (26.88↑)

the most consistent performance with only an 12% decrease, compared with the other approaches that showed a decrease of at least 40%. As the proportion of noise labels increased, the overly open strategy of adopting label information led to a stronger influence on the other related approaches, resulting in a significant reduction in the quality of data cleaning. MCRc-dc, on the other hand, reduced the dependence on labels through three different levels of constraints, which provided it with superior robustness.

3. *Of all the approaches, MCRc-dc was found to be the most statistically significant:* We conducted a T-test on the results of data cleaning. T-test is frequently employed to analyzing the significance of differences between two statistical sets. Tab. VI presents the T-test results of MCRc-dc compared to all other methods under a 90% noise level, where a larger t and a smaller p indicate a more significant difference. In all cases, the p -value was below 0.001, demonstrating a statistically significant deviation of MCRc-dc from the other methods. Given that the mean values of MCRc are consistently higher, it can be inferred that MCRc is statistically superior to the other methods.

C. Experimental Analysis of Robust Training

In this subsection, we will identify traffic and output a classifier based on its projected distance in the decision space. MCRc-clc was compared with several traffic identification models trained by other approaches, which were as follows.

- INCV, CL, FINE, and ULDC: Retrain a supervised classifier with the cleaned training set, and then identify the traffic in the validation set;
- MentorNet [13]: Select some pure samples to train a small guidance network to guide classifier training;
- Co-Teaching [14]: Train two networks with different initial states in parallel and make them select pure samples from each other;
- Co-Teaching+ [15]: Similar to Co-Teaching, but more inclined to select disagreement samples for another network;
- Co-Learning [16]: Train a two-headed encoder that avoids fitting noise labels to the network by means of consistency constraints on both heads.

TABLE VI
T-TEST RESULTS FOR MCRc-DC

Dataset	Scenario	sig.	INCV	CL	FINE	ULDC
Malicious_TLS	Asym	t	49.43	44.12	53.39	35.05
		p	<0.001	<0.001	<0.001	<0.001
	Sym	t	37.18	48.03	54.77	38.77
		p	<0.001	<0.001	<0.001	<0.001
CICIDS-2017	Asym	t	38.69	31.92	42.34	24.42
		p	<0.001	<0.001	<0.001	<0.001
	Sym	t	38.52	35.17	41.49	24.33
		p	<0.001	<0.001	<0.001	<0.001

TABLE VII
CLASSIFICATION ACCURACY IN ASYMMETRIC SCENARIOS ON MALICIOUS_TLS

Acc	10%	20%	30%	40%	50%	60%	70%	80%	90%
INCV	88.34±0.58	71.51±1.35	66.30±2.18	50.76±2.27	44.11±3.48	38.02±4.75	36.83±5.23	37.04±6.25	36.89±5.97
CL	87.53±1.02	72.32±2.24	67.14±3.17	54.08±3.36	49.35±4.69	43.64±4.62	31.82±5.27	25.90±5.57	22.44±6.62
FINE	80.54±1.33	74.23±1.71	75.67±1.95	61.39±2.79	45.42±2.74	37.13±3.67	26.29±4.62	20.01±4.66	12.16±5.35
ULDC	87.28±1.79	85.19±1.93	83.32±2.08	81.34±2.32	75.86±2.72	71.04±3.51	66.75±3.42	61.63±4.25	51.93±5.03
MentorNet	90.25±0.56	85.97±1.67	81.17±1.38	72.15±2.75	60.34±2.95	59.05±3.85	44.60±4.57	35.84±4.59	35.21±4.46
Co-Teaching	93.39±0.28	92.46±0.35	84.80±1.24	75.90±1.63	63.73±2.51	53.81±2.67	46.14±2.98	38.83±3.46	20.18±4.89
Co-Teaching+	94.63 ±0.32	92.32±0.89	90.60±1.11	87.31±1.44	68.93±2.83	41.27±3.01	40.05±3.39	36.76±4.36	36.73±4.42
Co-Learning	94.09±0.68	92.64±0.89	80.54±1.11	60.77±1.72	38.66±3.39	37.15±3.79	36.98±4.29	36.96±5.27	36.96±5.21
MCRc-cls (ours)	94.03	93.14	93.16	92.72	92.59	92.29	91.56	85.28	82.07
	±0.16 (0.60↓)	±0.37 (0.50↑)	±0.49 (2.56↑)	±0.96 (5.41↑)	±1.81 (16.73↑)	±1.93 (21.25↑)	±2.28 (24.81↑)	±3.50 (23.65↑)	±3.16 (30.14↑)

TABLE VIII
CLASSIFICATION ACCURACY IN SYMMETRIC SCENARIOS ON MALICIOUS_TLS

Acc	10%	20%	30%	40%	50%	60%	70%	80%	90%
INCV	85.81±0.51	46.46±3.89	39.26±3.75	40.19±3.74	40.87±4.04	40.68±5.38	36.95±5.16	36.21±5.35	37.00±6.28
CL	85.01±0.88	79.69±1.93	69.04±2.23	55.77±2.77	41.27±3.31	40.91±4.24	38.66±5.12	36.37±5.11	21.68±6.33
FINE	88.99±0.95	70.07±1.99	65.37±1.89	53.87±3.05	24.70±4.77	23.81±4.55	20.94±4.87	19.63±5.51	5.87±5.12
ULDC	86.12±1.18	82.17±2.05	80.79±2.27	77.03±3.03	75.20±4.37	66.19±5.16	65.22±5.21	59.20±5.17	50.26±6.16
MentorNet	90.26±0.42	87.27±0.85	79.92±2.04	71.51±2.62	59.39±3.95	40.67±3.79	38.69±4.66	37.05±4.68	36.30±5.95
Co-Teaching	94.90±0.32	88.62±1.21	81.48±1.36	72.21±1.59	60.95±2.62	47.17±2.79	34.26±3.01	21.11±4.91	7.05±5.54
Co-Teaching+	95.19±0.27	90.22±0.74	89.88±0.97	87.09±1.42	70.79±1.95	55.12±3.09	40.97±3.97	36.90±4.62	36.95±4.52
Co-Learning	95.72 ±0.18	94.48 ±0.93	90.49±0.91	71.40±1.65	49.47±2.73	38.01±3.22	36.98±4.69	36.92±5.01	36.84±5.95
MCRc-cls (ours)	94.30	94.14	93.15	92.55	91.63	91.31	90.48	87.93	84.76
	±0.06 (1.42↓)	±0.31 (0.34↓)	±0.36 (2.66↑)	±0.54 (5.46↑)	±0.88 (16.43↑)	±1.20 (25.12↑)	±1.92 (25.26↑)	±3.85 (28.73↑)	±3.93 (34.50↑)

Tab. VII, VIII, IX and X show the performance of MCRc-cl in asymmetric and symmetric scenarios on Malicious_TLS and CICIDS-2017, respectively. From the classification results, the following conclusions could be drawn:

1. At low noise levels ($\leq \text{asym-10\%}$ or $\leq \text{sym-20\%}$), MCRc-cl was only slightly lower at most only 1.4% than that of the best-performing approach: The design strategy of MCRc's careful acquisition of labels caused that its backbone acquired less information than the other supervised methods at low noise levels, which resulted in a slightly lower performance as well; this result was in line with our design expectations.

2. At high noise levels ($> \text{asym-10\%}$ or $> \text{sym-20\%}$), MCRc-cl performed significantly better than the other approaches: In particular, when noise levels reached 90% in both scenarios and datasets, MCRc-cl achieved the highest accuracy, which was 20% higher than the other approaches. The above results demonstrated the effectiveness of the multiple constraints designed in this study; this allowed the trained representation function to approximate the ideal function better and helped the classifier to identify malicious traffic under label noise conditions. We also assessed the statistical significance of MCRc-cl results compared to other methods at a noise level of 90%, as presented in Tab. XI. Similar to

TABLE IX
CLASSIFICATION ACCURACY IN ASYMMETRIC SCENARIOS ON CICIDS-2017

Acc	10%	20%	30%	40%	50%	60%	70%	80%	90%
INCV	87.75±1.03	77.08±1.65	69.06±2.79	59.64±2.78	48.38±3.13	39.19±4.07	38.05±4.63	36.66±5.04	36.72±5.48
CL	91.63±0.34	80.87±0.86	71.07±1.36	63.99±2.18	51.43±2.54	43.63±3.12	35.83±4.59	27.12±5.73	22.85±5.65
FINE	85.35±1.16	81.91±1.27	78.27±2.05	63.66±3.50	58.58±3.95	39.33±4.19	25.57±5.66	20.46±6.08	17.02±6.21
ULDC	91.28±0.71	83.93±0.91	87.15±1.92	81.94±2.90	81.34±2.93	78.30±3.56	80.96±4.19	75.88±5.13	69.05±5.05
MentorNet	93.93±0.85	90.43±1.12	80.64±2.02	68.78±3.64	53.97±3.13	47.01±4.19	40.04±4.48	37.94±5.95	32.35±5.71
Co-Teaching	98.38±0.11	98.03±0.17	90.69±0.99	83.55±1.93	75.98±1.56	59.09±3.19	41.46±4.13	29.88±5.90	21.06±5.87
Co-Teaching+	98.56 ±0.24	97.72±0.38	86.42±1.85	82.73±1.84	71.09±2.27	62.14±4.25	50.77±4.27	35.54±5.06	31.37±5.71
Co-Learning	98.28±0.14	97.37±0.47	84.15±0.85	78.92±1.77	64.72±2.29	50.98±4.03	46.65±4.81	38.76±4.97	30.92±5.79
MCRc-cls (ours)	98.20 ±0.07 (0.36↓)	98.05 ±0.25 (0.02↑)	97.94 ±0.40 (7.25↑)	97.87 ±1.00 (14.32↑)	97.62 ±1.17 (16.28↑)	97.59 ±2.84 (19.29↑)	96.91 ±2.66 (15.95↑)	96.32 ±3.73 (20.44↑)	92.01 ±6.11 (22.96↑)

TABLE X
CLASSIFICATION ACCURACY IN SYMMETRIC SCENARIOS ON CICIDS-2017

Acc	10%	20%	30%	40%	50%	60%	70%	80%	90%
INCV	87.31±0.94	81.94±1.73	76.38±2.33	62.75±3.49	49.23±4.58	48.81±5.52	40.29±6.05	35.63±5.77	37.71±5.98
CL	88.55±0.71	79.61±1.51	77.68±2.36	67.65±2.99	60.69±3.56	54.56±4.95	43.21±4.74	32.38±5.75	27.99±5.62
FINE	90.25±1.07	80.09±1.25	71.07±1.99	64.78±2.45	53.43±3.23	37.93±3.74	29.24±4.84	27.29±5.22	21.05±5.47
ULDC	88.53±0.76	81.00±1.05	84.92±2.39	77.64±2.73	68.60±4.05	65.58±4.17	61.51±4.91	58.67±5.47	55.26±6.02
MentorNet	96.85±0.18	88.23±1.55	80.78±1.36	71.93±2.85	63.35±3.82	56.69±4.05	49.95±4.17	38.94±4.47	37.87±5.73
Co-Teaching	96.28±0.59	90.24±0.94	85.02±1.31	71.19±1.77	65.75±2.58	60.68±2.44	52.94±3.35	43.34±4.09	26.05±4.77
Co-Teaching+	96.19±0.31	90.66±0.67	88.31±1.14	85.89±1.56	73.86±2.99	52.59±2.62	47.01±3.56	45.15±3.76	40.35±4.89
Co-Learning	98.51 ±0.16	98.85 ±0.19	92.46±0.53	85.07±1.19	84.32±1.81	71.75±2.88	52.45±3.67	45.17±3.98	40.74±5.16
MCRc-cls (ours)	98.13 ±0.14 (0.38↓)	98.07 ±0.31 (0.78↓)	97.90 ±0.71 (5.44↑)	97.96 ±0.94 (12.07↑)	97.49 ±1.14 (13.17↑)	96.51 ±1.35 (24.96↑)	95.37 ±2.96 (33.86↑)	94.96 ±3.58 (36.29↑)	90.58 ±5.15 (35.32↑)

MCRc-dc, MCRc-cls exhibited a p -value below 0.001 for all cases, demonstrating superior performance compared to other methods.

3. *MCRc-cls exhibited the best robustness to label noise:* Looking at each row of the table horizontally, the MCRc-cls decreased the least as the percentage of noise increased. In particular, when the noise level increased to 70%, the accuracy of MCRc-cls decreased by 4%. When the noise level increased to 90%, the accuracy of MCRc-cls was reduced by up to 18%, which was much smaller than that of the best-performing comparison approach. Due to the fact that MCRc-cls selected an unsupervised distance-based classifier that relied on representation functions rather than observed labels to classify traffic, the impact of noisy labels on classifier accuracy was mitigated. In addition, full confidence samples were constructed based on simple samples (rather than all samples), which also reduced the effect of noisy labels.

D. Ablation Analysis

In this subsection, we will check whether the three constraints designed for MCRc are necessary by ablation analysis. Specifically, we compared the performance of MCRc and its variants, including (1) the complete MCRc, (2) the MCRc that does not satisfy the information integrity

constraint, named MCRc~iic, (3) the MCRc that does not satisfy the cluster separability constraint, named MCRc~csc, and (4) the MCRc that does not satisfy the core proximity constraint, named MCRc~cpc.

Fig. 5 shows the performance of MCRc and its variants for six different scenes, including asym-0.2, asym-0.5, asym-0.8, sym-0.2, sym-0.5 and sym-0.8, where each corner of the radar plot represents a corresponding scene, and the center and edge correspond to 50% and 100%, respectively. By the difference in the area of the four hexagons, we can easily compare the importance of the three constraints and analyze the roles they play in different scenarios and tasks. We can draw the following conclusions:

1. *All three constraints were beneficial for noise traffic representation:* Of all the models, the complete MCRc achieved the best results in all scenarios and tasks. The proposed three constraints were designed to compress the value space of $\mathcal{G}$ in terms of information integrity, cluster separability and core proximity, driving the samples toward the correct cluster in the decision space, thus mitigating the effect of noisy labels.

2. *The core proximity constraint was the most helpful among all constraints for MCRc to tolerate noisy labels:* As the proportion of noisy labels increased, the performance of all three MCRc variants showed significant degradation of varying

TABLE XI
T-TEST RESULTS FOR MCR_e-CLS

Dataset	Scenario	sig.	INCV	CL	FINE	ULDC	MentorNet	Co-Teaching	Co-Teaching+	Co-Learning
Malicious_TLS	Asym	<i>t</i>	33.44	40.64	56.25	25.39	42.86	53.15	41.72	37.01
		<i>p</i>	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	Sym	<i>t</i>	32.23	42.33	61.11	23.60	33.98	57.20	39.91	33.60
		<i>p</i>	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
CICIDS-2017	Asym	<i>t</i>	33.68	41.53	43.03	14.48	35.67	41.86	36.25	36.28
		<i>p</i>	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	Sym	<i>t</i>	33.49	41.05	46.27	22.29	34.20	45.96	35.36	34.18
		<i>p</i>	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001

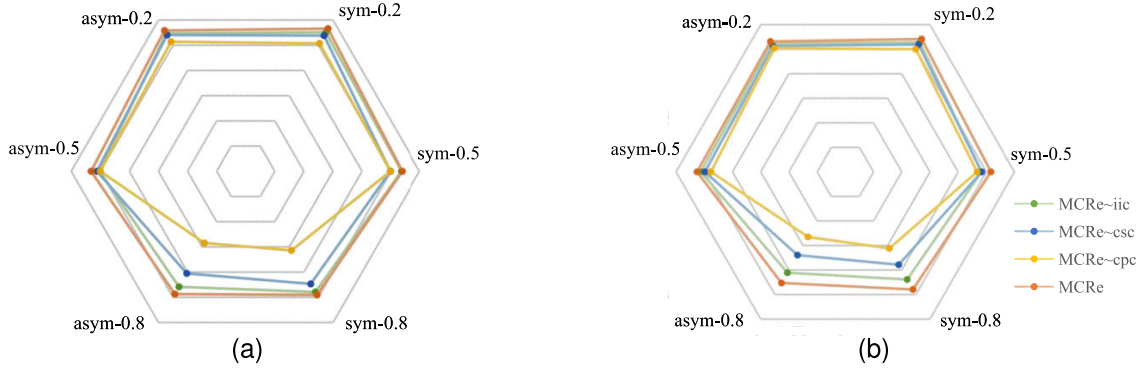


Fig. 5. Ablation results. (a) Percentage of pure samples in data cleaning tasks. (b) Accuracy in robust training tasks.

TABLE XII
COMPARISON OF MCR_e AND ITS VARIANTS

	Information Integrity Constraint	Cluster Separability Constraint	Core Proximity Constraint
MCR _e	✓	✓	✓
MCR _e ~iic	✗	✓	✓
MCR _e ~csc	✓	✗	✓
MCR _e ~cpc	✓	✓	✗

degrees. MCR_e cpc had the greatest degradation among all variations, with a classification accuracy of only about 70%. The results were in line with our design expectations, as $\mathcal{L}_{cpc}$ was only relevant to simple samples, and a conservative adoption strategy was used for labels to maintain higher classification accuracy in high-level noise scenarios.

To represent the effect of the three constraints on the directed optimization of the representation network more intuitively, we plotted Fig. 6 to show the process of MCR_e approximation to the ideal representation function during training. In particular, in this experiment, two high-level noise scenarios, asym-0.8 and sym-0.8, were selected. As presented in Fig. 6a, the accuracy of the model increased with epoch, and the representation network of MCR_e increasingly approximated the ideal representation function.

The binary cross-entropy curves between the original input and the output of the decoder are shown in Fig. 6b. The curve indicated changes in information loss during MCR_e representation, and the lower the entropy was, the smaller the information loss was, but the better the information integrity of the representation network was. As shown in Fig. 6b,

in both asym-0.8 and sym-0.8 scenarios, the information loss decreased with the epoch, which ensured better performance of MCR_e (see Fig. 6a).

Fig. 6c and Fig. 6d show the variation curves of the normalized mutual information (NMI) and adjusted rand index (ARI) of the traffic representations, respectively. The ARI and NMI have been commonly used as metrics for clustering algorithms. Their values closer to one indicate that clusters are more effective and can group more similar samples into the same clusters. In this experiment, the ARI and NMI were adopted to measure the cluster separability of MCR_e. As shown in Fig. 6c and Fig. 6d, the ARI and NMI gradually increased with epoch, proving that the representation model of MCR_e gradually converged and produced clusters with good separability.

The results in Fig. 6e show the trend of the average distance of the traffic to the corresponding full confidence samples in the $\mathbf{z}$ -space. The smaller the distance was, the closer the samples reconstructed by MCR_e were to the core of the class and the further they were from the decision boundary. In addition, as displayed in Fig. 6e, the mean distance showed a decreasing trend with the epoch increases, indicating that the projection of the traffic into the decision space was closer to the full confidence samples within the class, which helped MCR_e to identify mislabeling better.

E. Generalizability Analysis

In this subsection, we trained classifiers on several public malicious traffic datasets to evaluate the generalization of MCR_e. The results in Tab. II, III, VII, and VIII showed that the classification accuracy of MCR_e was positively correlated with data cleaning, so the generalization of MCR_e

TABLE XIII
CLASSIFICATION ACCURACY OF MCR-CLS ON DIFFERENT DATASETS IN ASYMMETRIC SCENARIOS

Acc (%)	10%	20%	30%	40%	50%	60%	70%	80%
NSL-KDD	97.46±0.09	97.11±0.22	96.68±0.56	96.09±0.84	94.32±1.88	93.96±2.77	93.53±2.95	92.59±3.38
UNSW-NB15	96.54±0.44	96.39±0.39	95.44±0.97	94.68±1.82	94.56±2.17	93.25±2.89	92.99±2.81	91.19±2.46
LITNET-2020	99.85±0.10	98.33±0.18	98.15±0.09	97.21±0.35	96.51±0.61	95.76±1.13	95.45±1.63	93.11±2.25
IOT-23	95.75±0.45	94.47±0.61	94.49±1.79	92.19±2.21	91.36±1.95	91.06±2.81	89.35±2.42	88.61±3.21
CICMalDroid-2020	94.92±0.85	93.59±0.93	93.08±1.32	92.52±1.53	91.16±1.99	90.82±2.14	88.26±2.86	87.47±2.98

TABLE XIV
CLASSIFICATION ACCURACY OF MCR-CLS ON DIFFERENT DATASETS IN SYMMETRIC SCENARIOS

Acc (%)	10%	20%	30%	40%	50%
NSL-KDD	97.55±0.18	97.47±0.32	95.76±1.05	94.94±1.53	90.13±1.58
UNSW-NB15	95.28±0.31	95.24±0.72	95.00±1.46	93.64±1.74	92.08±1.53
LITNET-2020	98.05±0.21	97.69±0.37	97.66±0.54	97.13±0.77	94.21±1.24
IOT-23	94.58±0.39	93.79±1.13	91.31±1.08	90.44±1.59	89.26±2.61
CICMalDroid-2020	94.18±0.35	93.91±0.53	92.75±1.14	90.52±1.22	88.06±1.36

TABLE XV
CL Vs. MCR+CL

Percentage of pure samples	Scenario	10%	20%	30%	40%	50%	60%	70%	80%	90%
CL	Asym	99.29 ±0.69	94.65±1.25	81.50±1.33	69.41 ±2.34	61.08±2.50	51.71 ±2.75	50.93±3.56	49.96 ±3.37	40.89±4.66
MCR+CL		95.93±1.08	95.71 ±1.41	95.65 ±1.54	95.03 ±1.47	95.09 ±1.83	94.95 ±2.07	94.32 ±2.61	92.97 ±2.79	88.28 ±3.70
CL	Sym	98.89 ±0.93	81.88±1.32	78.74±1.56	67.84±1.55	56.09±1.92	46.18±2.02	42.16±2.85	39.97±2.91	38.15±3.97
MCR+CL		95.97±1.03	95.80 ±1.18	95.35 ±1.22	94.98 ±1.75	94.77 ±2.26	94.22 ±2.71	93.73 ±3.18	93.21 ±3.06	87.99 ±3.45

TABLE XVI
CO-TEACHING VS. MCR+CO-TEACHING

Acc	Scenario	10%	20%	30%	40%	50%	60%	70%	80%	90%
Co-teaching	Asym	93.39 ±0.28	92.46 ±0.35	84.80 ±1.24	75.90 ±1.63	63.73±2.51	53.81±2.67	46.14±2.98	38.83±3.46	20.18±4.89
MCR+Co-teaching		83.39±1.18	80.23±1.77	74.40±1.69	70.39±1.97	68.24 ±2.05	55.09 ±2.04	49.68 ±2.86	41.98 ±2.93	36.97 ±3.59
Co-teaching	Sym	94.90 ±0.32	88.62 ±1.21	81.48 ±1.36	72.21 ±1.59	60.95±2.62	47.17±2.79	34.26±3.01	21.11±4.91	7.05±5.54
MCR+Co-teaching		84.33±0.95	82.79±1.21	77.38±1.49	68.76±1.91	65.71 ±1.99	49.49 ±2.23	39.06 ±2.17	36.94 ±2.95	35.26 ±3.36

could be evaluated based on by its classification accuracy. The datasets used in this subsection included: (1) traditional Internet traffic datasets such as NSL-KDD [34], UNSW-NB15 [35], and LITNET-2020 [37]; (2) IoT traffic datasets such as IoT-23 [38]; and (3) mobile Internet datasets such as CICMalDroid-2020 [39].

Tab. XIII and XIV show the accuracy of MCR on different datasets. Due to the small variety in some of the datasets, the upper limits for the proportion of noise in asymmetrical and symmetrical scenarios were set to 80% and 50%, respectively. Experimental results demonstrated that MCR achieved high accuracy on all datasets. In particular, MCR achieved an accuracy of higher than 87% when the proportion of asymmetric noise was as high as 80% or the proportion of symmetric noise was as high as 50%. The results showed that MCR had good generalisability and could identify not only encrypted malicious traffic, but also unencrypted and mixed malicious traffic.

F. Extensibility Analysis

In this subsection, we examined whether MCR has excellent extensibility. That is, whether MCR is compatible with other classical models, and enhances their data cleaning and traffic identification performance. Due to the excellent code integration of CL and Co-teaching, they were selected for validation. Specifically, the representation layer of MCR was plugged into the input layers of CL and Co-teaching respectively, and their output results were evaluated.

Tab. XV and XVI respectively show the results of MCR for the two models in different scenarios for the corresponding tasks. We can conclude as follows.

1. *MCR had good extensibility*: By simply concatenating models, MCR could be used in combination with other models, such as CL and Co-teaching, to process noise traffic. In fact, MCR could be considered as a data pre-processing process for other models. MCR represented raw traffic as a

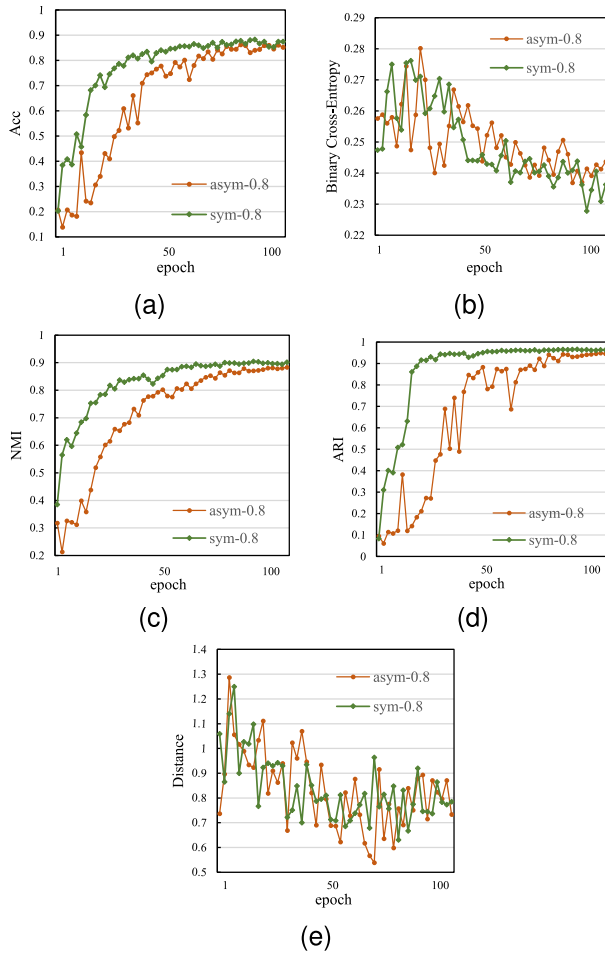


Fig. 6. Variations of MCRE metrics vs epoch. (a) Classification accuracy; (b) Binary cross-entropy; (c) Normalized mutual information; (d) Adjusted rand index; (e) Mean distance to the core of the class.

more robust feature vector and passed it on to other models for data cleaning or model training. In future work, we may consider encapsulating MCRE into a callable API to provide supports for other models more conveniently.

2. *At high noise levels, MCRE showed a significant enhancement to CL and Co-teaching:* The MCRE enhanced approaches were more effective than the original when the noise levels exceeded 20% and 50%, respectively. In particular, for CL, the percentage of pure data increased by more than 40% when the noise level was 90%; meanwhile, for Co-teaching, traffic was identified with a 16% improvement in accuracy. The above results indicated that robust traffic representation could help other models handle noise labels better, reduce reliance on label quality, ensure stable performance under high noise conditions, and output cleaner datasets or more accurate identification models.

VI. CONCLUSION

In this paper, a unified framework for data cleaning and robust training named MCRE is proposed for realistic malicious traffic. MCRE drove the representation network to approximate the ideal representation in iterations by applying information integrity constraints, cluster separability constraints and core proximity constraints. Compared to the state-of-the-art approaches, MCRE employs attribute-guided

training, which balances individual and global knowledge acquisition, resulting in a more robust performance ceiling for high-level label noise scenarios. Even at 90% noise labels, MCRE can achieve a pure sample percentage of 85% and a classification accuracy of 82%. In addition, MCRE is highly generalizable and capable of handling noise labels in various scenarios, including traditional and novel Internet scenarios. Finally, MCRE is also extensible, allowing connectivity to other data cleaning and robust training models and enhancing their performance.

In the future, we will focus on the application of MCRE in specific scenarios and special conditions, such as online environment detection, few-shot detection, and unknown attack detection.

ACKNOWLEDGMENT

The authors would like to thank the reviewers for their valuable comments that helped to improve this manuscript.

REFERENCES

- [1] N. Sambasivan, S. Kapania, and H. Highfill, "Everyone wants to do the model work, not the data work: Data cascades in high-stakes AI," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2021, pp. 1–15.
- [2] Z. Wang, K. W. Fok, and V. L. L. Thing, "Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study," *Comput. Secur.*, vol. 113, Feb. 2022, Art. no. 102542.
- [3] J. Zhang, F. Li, and F. Ye, "Autonomous unknown-application filtering and labeling for dl-based traffic classifier update," in *Proc. IEEE Conf. Comput. Commun.*, Jul. 2020, pp. 397–405.
- [4] X. Lin, G. Xiong, and G. Gou, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.
- [5] P. Lin, K. Ye, and Y. Hu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2022.
- [6] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [7] C. Dong, Z. Lu, Z. Cui, B. Liu, and K. Chen, "MBTree: Detecting encryption RATs communication using malicious behavior tree," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3589–3603, 2021.
- [8] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2017, pp. 1723–1732.
- [9] P. Chen, B. Liao, and G. Chen, "Understanding and utilizing deep neural networks trained with noisy labels," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 1062–1070.
- [10] C. Northcutt, L. Jiang, and I. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *J. Artif. Intell. Res.*, vol. 70, pp. 1373–1411, Apr. 2021.
- [11] T. Kim et al., "Fine samples for learning with noisy labels," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 24137–24149.
- [12] Q. Yuan et al., "ULDC: Unsupervised learning-based data cleaning for malicious traffic with high noise," *Comput. J.*, Apr. 2023, Art. no. bxad036, doi: [10.1093/COMJNL/BXAD036](https://doi.org/10.1093/COMJNL/BXAD036).
- [13] L. Jiang et al., "MentorNet: Learning data-driven curriculum for very deep neural networks on corrupted labels," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 2304–2313.
- [14] B. Han et al., "Co-teaching: Robust training of deep neural networks with extremely noisy labels," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 8536–8546.
- [15] X. Yu et al., "How does disagreement help generalization against label corruption?" in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7164–7173.
- [16] C. Tan et al., "Co-learning: Learning from noisy labels with self-supervision," in *Proc. 29th ACM Int. Conf. Multimedia*, 2021, pp. 1405–1413.

- [17] H. Song et al., "Learning from noisy labels with deep neural networks: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Mar. 7, 2022, doi: [10.1109/TNNLS.2022.3152527](https://doi.org/10.1109/TNNLS.2022.3152527).
- [18] H. Wei et al., "Open-set label noise can improve robustness against inherent label noise," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 7978–7992.
- [19] N. Karim et al., "UNICON: Combating label noise through uniform selection and contrastive learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 9676–9686.
- [20] A. Fahad, A. Almalawi, Z. Tari, K. Alharthi, F. S. Al Qahtani, and M. Cheriet, "SemTra: A semi-supervised approach to traffic flow labeling with minimal human effort," *Pattern Recognit.*, vol. 91, pp. 1–12, Jul. 2019.
- [21] Z. Yang et al., "WTAGRAPH: Web tracking and advertising detection using graph neural networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 1540–1557.
- [22] M. Kim and I. Lee, "Human-guided auto-labeling for network traffic data: The GELM approach," *Neural Netw.*, vol. 152, pp. 510–526, Aug. 2022.
- [23] J. Zhang et al., "Autonomous unknown-application filtering and labeling for DL-based traffic classifier update," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Jun. 2020, pp. 397–405.
- [24] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *Proc. 31st USENIX Secur. Symp. (USENIX Security)*, 2022, pp. 2783–2800.
- [25] W. Li, X.-Y. Zhang, H. Bao, H. Shi, and Q. Wang, "ProGraph: Robust network traffic identification with graph propagation," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1385–1399, Jun. 2023.
- [26] Z. Zhu, Z. Dong, and Y. Liu, "Detecting corrupted labels without training a model to predict," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 27412–27427.
- [27] R. Wang, T. Liu, and D. Tao, "Multiclass learning with partially corrupted labels," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 6, pp. 2568–2580, Jun. 2018.
- [28] E. Arazo et al., "Unsupervised label noise modeling and loss correction," in *Proc. Int. Conf. Mach. Learn. (PMLR)*, 2019, pp. 312–321.
- [29] D. Arpit et al., "A closer look at memorization in deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 233–242.
- [30] A. Iscen et al., "Learning with neighbor consistency for noisy labels," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 4672–4681.
- [31] D. Ortego et al., "Multi-objective interpolation training for robustness to label noise," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2021, pp. 6606–6615.
- [32] H. Maennel et al., "What do neural networks learn when trained with random labels?" in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 19693–19704.
- [33] K. He et al., "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9729–9738.
- [34] L. Dhanabal and S. P. Shantharajah, "A study on NSL-KDD dataset for intrusion detection system based on classification algorithms," *Int. J. Adv. Res. Comput. Commun. Eng.*, vol. 4, no. 6, pp. 446–452, 2015.
- [35] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1–6.
- [36] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.
- [37] R. Damasevicius et al., "LITNET-2020: An annotated real-world network flow dataset for network intrusion detection," *Electronics*, vol. 9, no. 5, p. 800, May 2020.
- [38] A. Parmisano, S. Garcia, and M. Erquiaga. (2020). *A Labeled Dataset With Malicious and Benign IoT Network Traffic*. [Online]. Available: <https://mcfp.felk.cvut.cz/publicDatasets/IoT-23-Dataset>
- [39] D. S. Keyes et al., "EntropLyzer: Android malware classification and characterization using entropy analysis of dynamic characteristics," in *Proc. Reconciling Data Analytics, Autom., Privacy, Secur., Big Data Challenge (RDAAPS)*, 2021, pp. 1–12.
- [40] K. M. Al-Gethami, M. T. Al-Akhras, and M. Alawairdhi, "Empirical evaluation of noise influence on supervised machine learning algorithms using intrusion detection datasets," *Secur. Commun. Netw.*, vol. 2021, pp. 1–28, Jan. 2021.



Qingjun Yuan received the M.Eng. degree from Strategic Support Force Information Engineering University, China, in 2016. He is currently with the Henan Key Laboratory of Network Cryptography Technology. His research interests include network security and side channel attack.



Gaopeng Gou received the bachelor's, M.Eng., and Ph.D. degrees from Beihang University in 2005, 2008, and 2014, respectively. He is currently a Full Professor with the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include network security and network anomaly detection.



Yanbei Zhu received the M.A.S. degree from the Beijing Institute of Technology, China, in 2019. She is currently with the Henan Key Laboratory of Network Cryptography Technology. Her research interests include network security and complex data analysis.



Yuefei Zhu received the Ph.D. degree from the Zhengzhou Information Science Technology Institute, China, in 1990. He is currently a Professor with the Henan Key Laboratory of Network Cryptography Technology, China. He is the Main Designer of the Elliptic Curve Cryptography (ECC) Public-Key Algorithm SM2. His research interests include cryptography and network security.



Gang Xiong is currently a Full Professor and a Ph.D. Supervisor with the Institute of Information Engineering, Chinese Academy of Sciences, China. He has authored more than 60 papers in refereed journals and conference proceedings. His research interests include network and information security. He is a member of the 3rd Communication Security Technical Committee of the China Institute of Communications.



Yongjuan Wang received the Ph.D. degree from Information Engineering University in 2009. She is currently with the Henan Key Laboratory of Network Cryptography Technology. Her main research interests include cryptographic analysis and cyberspace security.